

SDD para projetos com agentes de programação

Guia autônomo de especificação planejamento e tarefas

Material do estudante

Sistemas Inteligentes Aplicados

Uso individual ou em grupo

Este guia mostra como transformar uma necessidade em especificação, plano e tarefas. O material pode ser usado com arquivos Markdown comuns ou com uma ferramenta compatível com Spec Kit.

Ao terminar, produza uma pasta de funcionalidade que outra pessoa consiga revisar, implementar e verificar sem depender de explicação oral.

Conteúdo

As seções seguem o trabalho de uma funcionalidade, da definição do problema até a comparação entre artefatos e implementação.

- 1** O que muda com SDD
- 2** Os três artefatos centrais
- 3** Fluxo de trabalho
- 4** Estrutura da funcionalidade
- 5** Como escrever a especificação
- 6** Como revisar a especificação
- 7** Como produzir o plano
- 8** Como registrar pesquisa técnica
- 9** Como criar tarefas executáveis
- 10** Como manter rastreabilidade
- 11** Como trabalhar com agentes de programação
- 12** Como lidar com mudanças
- 13** Roteiro de trabalho do grupo
- 14** Erros frequentes
- 15** Critérios para considerar os artefatos prontos
- 16** Glossário

1 O que muda com SDD

Em um fluxo improvisado, a equipe descreve uma ideia em uma mensagem, pede código e decide os detalhes durante a implementação. O resultado pode executar e ainda resolver o problema errado. Também fica difícil descobrir por que uma regra existe, qual cenário deveria passar e se uma alteração mudou o comportamento combinado.

No SDD, a especificação permanece como referência. Ela define o que deve acontecer e por que esse comportamento importa. O plano toma decisões de arquitetura e tecnologia para realizar a especificação. As tarefas transformam o plano em unidades de trabalho executáveis e verificáveis.

```
intenção -> especificação revisada -> plano revisado -> tarefas -> implementação -> convergência
```

O processo não elimina revisão humana. Um agente pode redigir arquivos, localizar inconsistências e implementar tarefas, mas não sabe quais decisões o PO confirmou. Quando o contexto está incompleto, uma saída detalhada pode apenas esconder uma suposição.

SDD não significa documentação extensa

O tamanho do artefato deve acompanhar o risco e a complexidade. Uma mudança pequena pode usar uma spec, um plano curto e poucas tarefas. Uma funcionalidade com dados, integrações e risco de erro precisa registrar contratos, modelo de dados, casos de falha e decisões técnicas.

Um arquivo grande não é melhor por si só. O artefato é útil quando outra pessoa consegue tomar uma decisão, implementar ou verificar o comportamento sem depender de uma explicação oral.

2 Os três artefatos centrais

Artefato	Pergunta principal	Conteúdo	Evite
spec.md	O que deve acontecer e por quê	usuário, problema, escopo, histórias, requisitos, critérios e riscos	framework, banco e estrutura de classes
plan.md	Como a equipe pretende construir	arquitetura, dependências, dados, contratos, testes e operação	alterar silenciosamente o escopo
tasks.md	Em que ordem o trabalho será executado	ações, caminhos, dependências, histórias e verificações	itens vagos como “fazer backend”

Os arquivos se relacionam, mas não são cópias. Se a spec exige uma fonte verificável, o plano define como os metadados atravessam o sistema. As tarefas criam modelo, teste, serviço e interface necessários para exibir essa fonte.

Artefatos de apoio

`constitution.md` registra princípios estáveis do projeto. `research.md` resolve dúvidas técnicas e compara alternativas. `data-model.md` define entidades e regras do domínio. `contracts/` fixa interfaces observáveis. `quickstart.md` ensina outra pessoa a executar e validar o incremento. `checklists/` contém revisões da qualidade dos requisitos. `traceability.md` liga requisitos, testes e tarefas.

Não crie arquivos vazios apenas para completar uma estrutura. Registre “não aplicável” com uma justificativa ou remova o artefato que não ajuda a funcionalidade.

3 Fluxo de trabalho

O fluxo central do Spec Kit segue Specify, Plan, Tasks, Implement e Converge. As etapas Clarify, Checklist e Analyze funcionam como revisões intermediárias. A constituição estabelece regras do projeto antes das funcionalidades.

Constituição

A constituição contém princípios que devem valer em várias funcionalidades. Exemplos incluem proibição de dados pessoais reais, confirmação antes de ação externa, exigência de fonte para afirmação factual e testes mínimos para uma entrega.

Evite colocar detalhes locais na constituição. “A tela de consulta usa um botão azul” pertence à funcionalidade ou ao sistema de design. “Toda ação externa exige confirmação humana” pode ser um princípio do projeto.

Especificar

A especificação descreve o problema, o resultado e o comportamento. Escreva sem escolher tecnologia. Uma pessoa de produto deve conseguir revisar o documento sem conhecer o framework.

Esclarecer

O esclarecimento busca decisões que mudam escopo, comportamento, risco ou aceitação. Faça poucas perguntas de alto impacto. Dúvidas técnicas que não alteram o produto seguem para `research.md`.

Revisar requisitos

O checklist avalia o texto da especificação. Ele pergunta se os requisitos são completos, claros, consistentes e verificáveis. Marcar um item não significa que o código está pronto.

Planejar

O plano seleciona arquitetura e componentes depois que a spec está estável. Registre as escolhas, o motivo e a evidência prevista. Se a solução técnica não consegue atender um requisito, volte à discussão em vez de apagar ou enfraquecer o requisito.

Criar tarefas

As tarefas são organizadas por dependência e história de usuário. A história P1 deve produzir um incremento demonstrável antes das prioridades posteriores sempre que possível.

Analisar

A análise compara spec, plano e tarefas. Ela localiza requisitos sem cobertura, tarefas sem origem, termos contraditórios e lacunas de teste. Essa revisão ocorre antes da implementação, quando corrigir um documento ainda custa pouco.

Implementar

A implementação segue as tarefas revisadas. Uma tarefa é marcada como concluída depois de o resultado ser verificado. Código criado não equivale a comportamento aprovado.

Convergir

A convergência compara o estado final com os artefatos. Lacunas viram tarefas explícitas. O ciclo termina quando os critérios possuem evidência ou quando uma pendência foi aceita e registrada pelo responsável.

Os nomes e a forma de invocar comandos variam entre integrações do Spec Kit. Os arquivos deste kit funcionam mesmo sem a ferramenta.

4 Estrutura da funcionalidade

Use uma pasta numerada e um nome curto.

```
specs/001-consulta-prazos/
├── spec.md
├── plan.md
├── research.md
├── data-model.md
├── quickstart.md
├── tasks.md
├── traceability.md
├── checklists/
│   └── requirements.md
└── contracts/
    └── openapi.yaml
```

O número fornece uma referência estável. O nome ajuda a localizar a funcionalidade. Não reutilize a mesma pasta para comportamentos sem relação apenas porque usam o mesmo componente técnico.

5 Como escrever a especificação

Comece pelo problema

Identifique a pessoa, a situação, a barreira observada e o resultado esperado. Compare os exemplos.

Frase	Avaliação
“Criar uma IA moderna para melhorar o atendimento”	não identifica usuário, situação ou resultado observável
“Estudantes que não conhecem os setores precisam identificar onde tratar uma solicitação”	define pessoa, dificuldade e resultado

A alternativa atual ajuda a estabelecer uma linha de base. Se o estudante consulta uma página de setores em quatro minutos, uma nova solução pode ser comparada com esse fluxo.

Delimite o escopo

O escopo incluído descreve a menor jornada que entrega valor. O não escopo impede que uma intenção pequena se transforme em um sistema ilimitado.

Incluído

- sugerir categoria e setor
- permitir confirmação ou correção
- tratar baixa confiança

Fora do escopo

- enviar a solicitação
- consultar dados individuais
- aprender automaticamente com a correção

O não escopo não é uma lista de ideias sem prioridade. Inclua itens que alguém poderia presumir como parte da funcionalidade e que precisam de uma fronteira explícita.

Escreva histórias independentes

Uma história descreve pessoa, ação e resultado. A prioridade representa valor e ordem de entrega, não dificuldade técnica.

Como estudante, quero perguntar por um prazo acadêmico para localizar a data e conferir a fonte oficial.

O teste independente explica como demonstrar a história sem depender das posteriores. Se a P1 só funciona depois de todas as outras histórias, a divisão provavelmente foi feita por camada técnica, e não por valor.

Use cenários observáveis

Um cenário de aceitação contém estado inicial, evento e resultado.

Dado que o corpus não contém evidência suficiente
Quando o estudante pergunta por uma taxa
Então o sistema declara a limitação e não apresenta um valor como fato

O cenário não precisa descrever cada clique. Ele define o comportamento que decide aceitação. O teste pode ser manual, automatizado ou uma combinação, conforme o risco.

Escreva requisitos específicos

Cada requisito recebe um identificador e um verbo verificável.

Requisito fraco	Requisito verificável
“O sistema deve ser rápido”	“A resposta fica disponível em até três segundos no percentil 95 do teste definido”
“A IA deve responder corretamente”	“Toda afirmação factual apresenta fonte que contém a informação”
“A interface deve ser intuitiva”	“Quatro de cinco participantes concluem a tarefa em até dois minutos sem ajuda”
“O sistema deve ser seguro”	“A pergunta bruta e dados pessoais detectados não aparecem nos logs”

Evite incluir a solução no requisito. “Usar PostgreSQL” é decisão de plano. “Persistir a versão da fonte usada” é comportamento ou requisito de dados.

Defina sucesso e qualidade

Critérios de sucesso medem o resultado da funcionalidade. Requisitos de qualidade definem propriedades como desempenho, acessibilidade, privacidade e confiabilidade.

Informe conjunto, unidade, limiar e avaliador quando forem necessários. “Acertar 12 de 15 casos revisados pelo PO” é mais verificável que “alta precisão”. Um conjunto pequeno serve ao exercício, mas não comprova desempenho em produção.

Registre dúvidas sem inventar respostas

Uma questão aberta deve informar quem decide e quando a resposta é necessária. Se ela impede comportamento ou escopo, não aprove a spec. Se é uma configuração ainda sem valor final e o plano suporta configurá-la, registre a pendência como não bloqueadora.

6 Como revisar a especificação

Leia a spec como se você não tivesse participado da conversa. Tente executar mentalmente um caso de sucesso, um sem informação e um de falha.

Pergunte:

- o usuário e o resultado estão claros;
- a jornada termina em valor observável;
- limites e exceções possuem comportamento;
- os termos importantes têm definição;
- histórias e requisitos não se contradizem;
- cada critério pode ser aceito ou rejeitado;
- suposições estão separadas de decisões.

O checklist de requisitos pertence ao revisor. Um agente pode apontar problemas, mas a aprovação permanece humana. No exemplo de triagem deste kit, a primeira versão usa expressões como “qualquer solicitação”, “rápida” e “funcionar bem”. A revisão transforma essas frases em categorias, limites, casos e medidas.

7 Como produzir o plano

O plano começa pela spec aprovada e pela constituição. Ele deve explicar a abordagem técnica em termos suficientes para orientar tarefas e revisão.

Contexto técnico

Registre linguagem, versões, dependências, persistência, testes, plataforma, limites e escala. Use “precisa de esclarecimento” quando uma decisão realmente bloquear o plano. Não escolha uma tecnologia apenas porque ela apareceu na descrição inicial.

Verificação da constituição

Para cada princípio, mostre a decisão correspondente e a evidência prevista. Se uma exceção for necessária, registre motivo, risco e alternativa rejeitada. Exceções silenciosas anulam o valor da constituição.

Arquitetura e responsabilidades

Um diagrama textual simples pode ser suficiente.

```
navegador -> API -> serviço de domínio -> recuperador -> índice
                                     |-> gerador de resposta
                                     |-> registro da execução
```

Depois do desenho, explique quem valida entrada, quem decide estados, quem acessa dados e quem formata a saída. Interfaces ajudam quando um componente pode mudar, mas não crie camadas sem responsabilidade concreta.

Dados e contratos

O modelo de dados descreve entidades, campos e regras do domínio. O contrato define entradas, saídas, erros e versionamento de uma interface. Eles devem concordar com a spec.

Se a spec exige documento, seção e versão, o contrato de resposta e o modelo de evidência precisam transportar esses campos. Uma citação montada no frontend a partir de texto livre perde rastreabilidade.

Estratégia de testes

Planeje testes conforme o comportamento:

- unitários para regras isoladas;
- integração para banco, busca e serviços;
- contrato para interfaces;

- jornada para o fluxo do usuário;
- avaliação para comportamento variável de IA.

Os testes não devem medir apenas o texto final. Em sistemas com IA, verifique estado, evidência, ferramenta chamada, argumentos, custo e duração quando forem relevantes.

Operação

Considere configuração, segredos, logs, timeout, retomada e saúde do serviço. Um protótipo publicado precisa indicar como falha e como outra pessoa confirma seu estado.

8 Como registrar pesquisa técnica

Use `research.md` quando houver alternativas reais. Cada decisão deve conter questão, critérios, opções, escolha, justificativa, consequências e condição de revisão.

Questão: qual mecanismo de recuperação atende ao primeiro incremento
 Critérios: execução local, corpus pequeno e inspeção do resultado
 Alternativas: busca textual, embeddings remotos e contexto completo
 Decisão: busca textual atrás de uma interface
 Condição para revisão: falha acima do limite no conjunto de avaliação

“Escolhemos porque é melhor” não explica a decisão. Cite documentação oficial, experimento ou restrição do projeto. Uma condição de revisão impede que uma escolha provisória vire regra permanente sem reavaliação.

9 Como criar tarefas executáveis

Uma tarefa deve permitir que outra pessoa saiba o que alterar e como verificar o resultado.

- [] T019 [P] [US2] Criar teste de ausência em
 backend/tests/unit/test_query_service.py

O identificador facilita referências. [US2] liga a tarefa à história. [P] significa paralelismo real: a tarefa não depende de trabalho pendente e não disputa os mesmos arquivos.

Compare:

Tarefa fraca	Tarefa executável
“Fazer backend”	“Implementar POST /api/v1/queries em backend/src/api/routes/queries.py”
“Adicionar testes”	“Criar teste do estado insufficient_evidence em backend/tests/contract/test_queries_api.py”
“Melhorar segurança”	“Impedir pergunta bruta nos logs e verificar em backend/tests/integration/test_query_log.py”

Ordem recomendada

1. Preparação do projeto.

2. Base compartilhada que bloqueia todas as histórias.
3. História P1 com teste independente.
4. Histórias posteriores em prioridade.
5. Qualidade transversal, avaliação e documentação final.

Evite organizar tarefas apenas em “backend”, “frontend” e “banco”. Essa divisão costuma deixar todas as histórias parcialmente prontas e nenhuma demonstrável.

Testes nas tarefas

Quando a spec exige teste, a lista deve incluir a criação e a execução correspondente. Escrever um arquivo de teste não encerra a tarefa de verificação. Registre a evidência no local combinado pelo projeto.

10 Como manter rastreabilidade

A matriz de rastreabilidade liga as camadas do trabalho.

Requisito	Cenário	Teste	Tarefas
RF 004 ausência de evidência	US2 cenário 1	test_insufficient_evidence	T019 T020 T021

Uma linha vazia revela uma lacuna. Requisito sem teste pode ser impossível de verificar. Tarefa sem requisito pode introduzir escopo. Teste sem cenário pode validar uma decisão que ninguém aprovou.

A matriz não precisa repetir todos os detalhes. Use identificadores e links para manter a navegação.

11 Como trabalhar com agentes de programação

Forneça o artefato certo para cada etapa. Durante a especificação, evite pedir arquitetura. Durante o plano, inclua a spec e a constituição. Para tarefas, inclua todos os artefatos de design relevantes.

Dê limites ao pedido

Um bom pedido define arquivo de entrada, saída, decisões proibidas e forma de revisão.

```
Revise spec.md usando checklists/requirements.md.
Não altere decisões de produto.
Liste ambiguidades e proponha perguntas ao P0.
```

Revise antes de avançar

Não execute specify, plan, tasks e implement como uma sequência sem leitura. Um erro na spec se espalha para todos os arquivos posteriores e pode parecer mais confiável por aparecer repetido.

Preserve autoria e evidência

Registre prompt, contexto, saída aceita, alterações humanas, testes e commit conforme as regras da disciplina. Não é necessário registrar raciocínio interno oculto do modelo. O que importa é permitir que outra pessoa entenda o que foi usado e como o grupo verificou o resultado.

Não delegue decisões do PO

Quando um requisito admite alternativas que mudam valor, risco ou experiência, peça decisão. O agente pode estruturar opções e consequências. Ele não deve transformar ausência de resposta em aprovação.

12 Como lidar com mudanças

Requisitos mudam. A equipe precisa decidir como os artefatos permanecem válidos. Três abordagens comuns são:

- `spec first`: a spec orienta a construção inicial e depois perde autoridade;
- `spec anchored`: a spec continua como referência e é atualizada com mudanças aprovadas;
- `spec as source`: a spec é a fonte editada por humanos e outros artefatos são derivados dela.

Para a disciplina, use o modelo ancorado. Preserve a spec, registre mudanças de comportamento e atualize plano, tarefas, testes e contratos afetados. Não reescreva o histórico para fingir que a decisão sempre foi a mesma.

Uma mudança técnica que preserve comportamento pode atualizar plano e pesquisa sem alterar a spec. Uma mudança no que o usuário vê, nos limites ou nos critérios de aceitação exige revisão da spec e nova confirmação quando aplicável.

13 Roteiro de trabalho do grupo

1. Escolha uma jornada pequena do backlog.
2. Copie a pasta `modelos` para `specs/NNN-nome-curto`.
3. Preencha `spec.md` e revise com o checklist.
4. Resolva questões de produto com o PO.
5. Pesquise dúvidas técnicas e produza o plano.
6. Revise constituição, dados, contratos e testes.
7. Gere tarefas e analise consistência.
8. Implemente a P1 e execute seu teste independente.
9. Entregue histórias posteriores sem quebrar a P1.
10. Compare o resultado com os artefatos e registre lacunas.

O arquivo `atividades/roteiro_autonomo.md` apresenta saídas esperadas para cada etapa. `atividades/prompts_para_agente.md` contém pedidos que podem ser adaptados a diferentes agentes.

14 Erros frequentes

A spec escolhe tecnologia

Mova framework, banco, biblioteca e arquitetura para o plano. Mantenha na spec somente uma restrição tecnológica realmente confirmada pelo projeto.

O plano muda o requisito

Não reduza silenciosamente um critério porque a implementação é difícil. Registre a incompatibilidade e volte à decisão de produto.

As tarefas são camadas genéricas

Reorganize por história e inclua teste independente. O objetivo é concluir valor, não acumular componentes parciais.

O checklist testa o código

Use o checklist de requisitos para avaliar o texto da spec. Testes de código aparecem no plano e nas tarefas.

Tudo recebe prioridade P1

Escolha a menor jornada que entrega valor. Prioridade sem escolha não orienta execução.

Checkboxes são marcados antes da verificação

Uma tarefa concluída precisa de resultado observável. Se o código existe, mas o teste falha ou não foi executado, mantenha a tarefa aberta e registre o bloqueio.

15 Critérios para considerar os artefatos prontos

Uma spec está pronta para planejamento quando outra pessoa entende o problema, consegue percorrer sucesso e falhas, e pode aceitar ou rejeitar cada comportamento.

Um plano está pronto para tarefas quando decisões técnicas bloqueadoras foram resolvidas, componentes e contratos concordam com a spec, e a estratégia de teste cobre os riscos principais.

As tarefas estão prontas para implementação quando possuem ordem, caminhos, dependências, histórias e resultados verificáveis. A lista precisa permitir uma primeira entrega útil antes de todo o projeto terminar.

16 Glossário

Critério de aceitação: condição observável usada para aceitar ou rejeitar um comportamento.

Constituição: conjunto de princípios estáveis que limita decisões do projeto.

Contrato: descrição observável de entrada, saída, erro e versão de uma interface.

Convergência: comparação final entre implementação e artefatos para localizar lacunas.

História de usuário: descrição curta de pessoa, ação e resultado.

PO: pessoa responsável por prioridades e decisões de produto.

Requisito funcional: comportamento que o sistema deve apresentar.

Requisito de qualidade: propriedade verificável de desempenho, segurança, acessibilidade, confiabilidade ou outra dimensão.

SDD: desenvolvimento em que especificações orientam planejamento, implementação e verificação.

Tarefa: unidade de trabalho com ação, localização e resultado verificável.

Teste independente: forma de demonstrar uma história sem depender das histórias posteriores.

Rastreabilidade: ligação entre requisito, cenário, teste, tarefa e implementação.

Referências

- GitHub Spec Kit. Documentação inicial. <https://github.com/github/spec-kit/blob/main/docs/index.md>
- GitHub Spec Kit. Guia rápido de SDD. <https://github.com/github/spec-kit/blob/main/docs/quickstart.md>
- GitHub Spec Kit. Conceitos de SDD. <https://github.com/github/spec-kit/blob/main/docs/concepts/sdd.md>
- GitHub Spec Kit. Modelos de persistência da especificação. <https://github.com/github/spec-kit/blob/main/docs/concepts/spec-persistence.md>
- GitHub Spec Kit. Templates de spec, plano e tarefas. <https://github.com/github/spec-kit/tree/main/templates>

Documentação consultada em 24 de setembro de 2026. Os comandos da ferramenta podem mudar; os princípios e modelos deste kit permanecem utilizáveis como arquivos Markdown comuns.